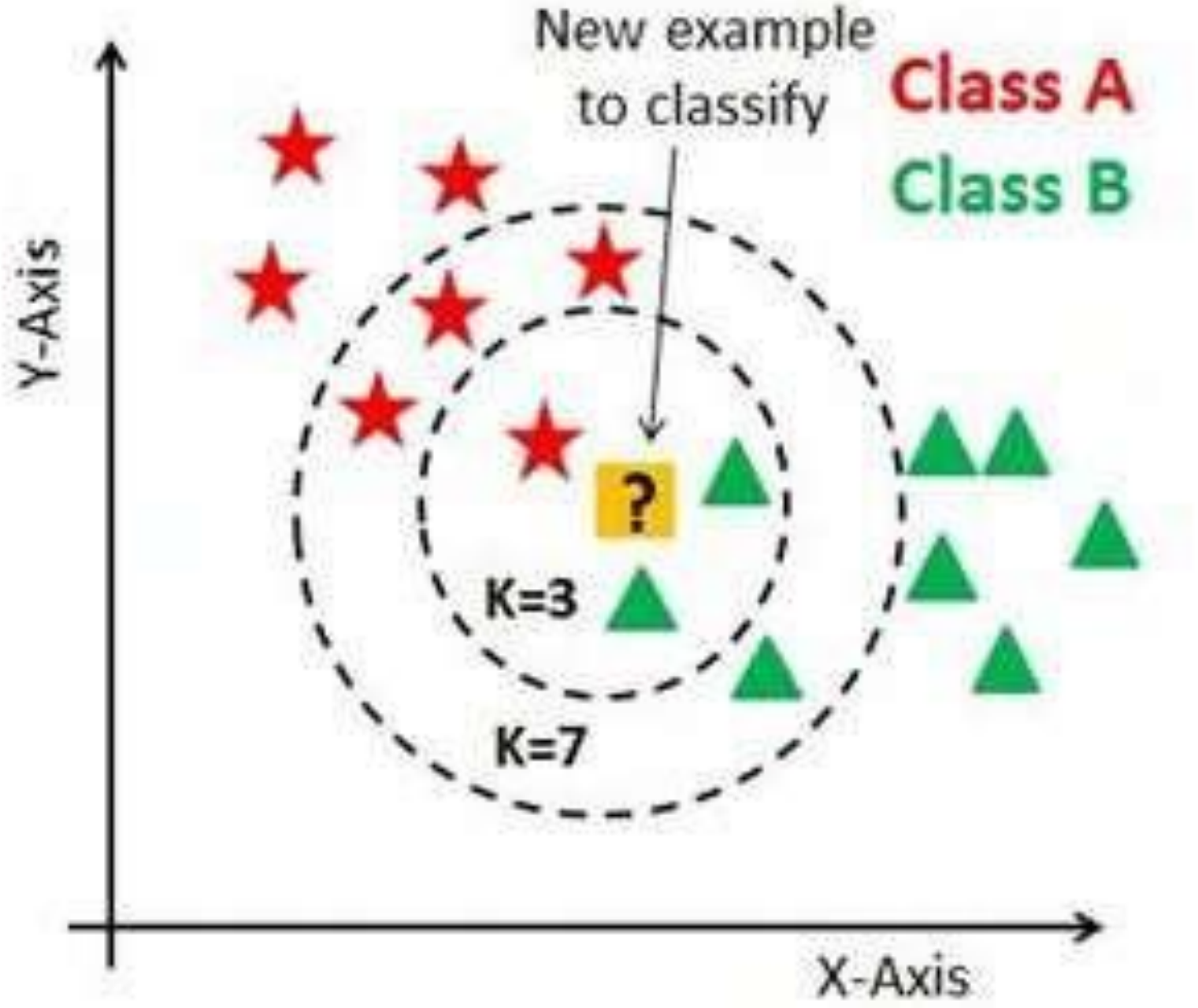
Instance-Based ***Learning in Machine Learning***

- **Historical Context**: Instance-based learning has been foundational in pattern recognition and classification tasks since the early days of AI.The k-NN algorithm, one of the simplest and most effective instance-based methods, was developed in the 1950s.
- **Definition**: Instance-based learning, also known as memory-based learning or lazy learning, is a type of machine learning where the algorithm stores instances of training data and uses them to make predictions on new data. Instead of explicitly creating a model during the training phase, instance-based learning methods delay the generalization process until a query is made during the prediction phase.


New example
to classify
Class A
Class B
Y-Axis
?
K=3
K=7
X-Axis

Key Characteristics:

- **LazyLearning**: No explicit model is built during training. The algorithm simply stores the training instances. Generalization occurs at the time of prediction.

- **Similarity-Based**: Predictions are made based on the similarity between the new data instance and stored training instances. Common similarity measures include Euclidean distance, Manhattan distance, and cosine similarity.

- **Local Decision Making**: Decisions are made based on the local neighbourhood of the input instance. The algorithm considers only a subset of the training data (e.g., the k-nearest neighbours) to make a prediction.

Example Algorithms:

- ***k-Nearest Neighbors (k-NN):***
  - **Algorithm**: For a given input instance, the algorithm finds the k training Instances closest to the input and assigns the most common label (for classification) or averages the labels (for regression).
  - **Application**:Used in recommendation systems, image recognition, and Anomaly detection.

Advantages of Instance-Based Learning:

- ***Simplicity:***
  - Easy to implement and understand.
  - No need for a complex model training phase.
- *Adaptability:*
  - Can adapt quickly to new data without retraining the entire model.
  - Useful for applications where the data distribution changes overtime.
- *Performance:*
  - Effectiveforproblemswherethedecisionboundaryisirregularandcannot be easily captured by a global model.

Disadvantages of Instance-Based Learning:

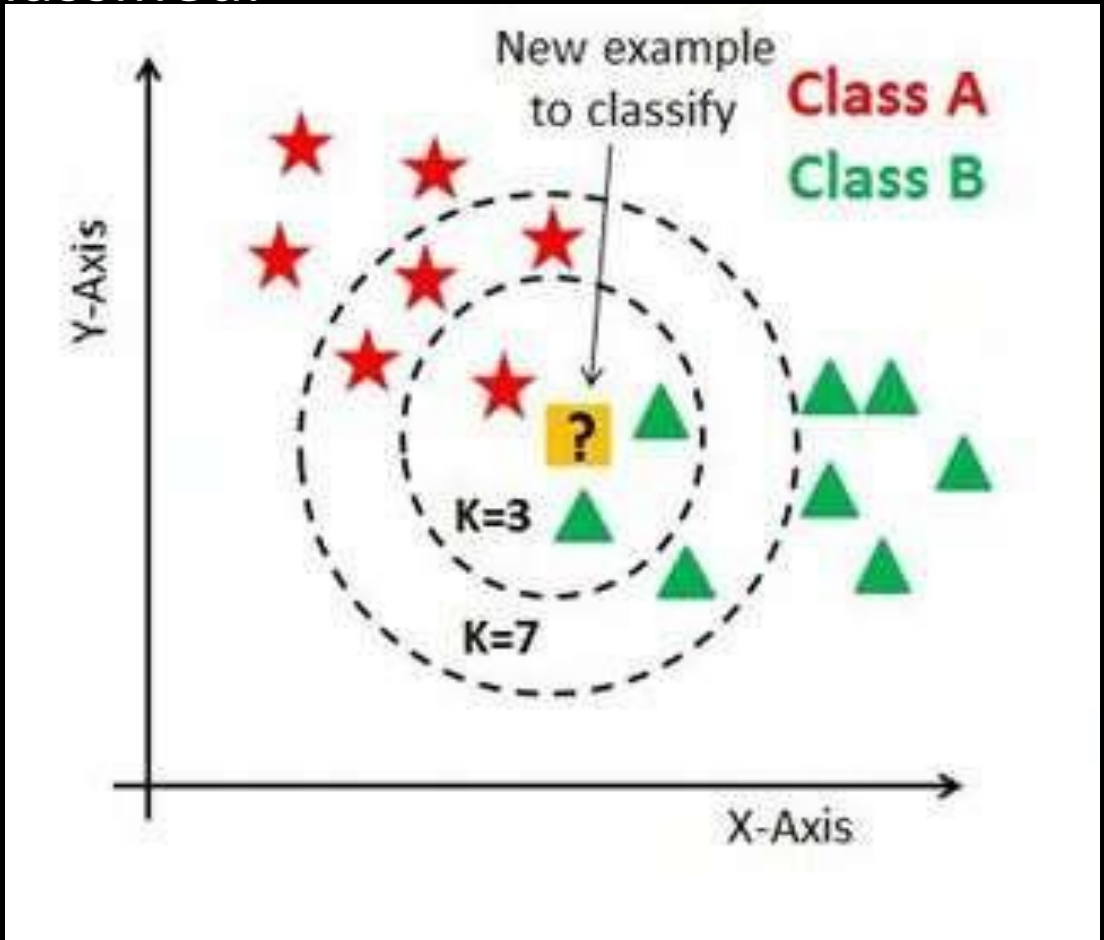
- *Computational Cost*:
  - High storage requirements since all training instances are retained.
  - Prediction can be slow for large data sets due to the need to compute distances to all stored instances.
- *Sensitivity to Noise*:
  - Can be sensitive to noisy data,as outliers can affect the prediction.
- *Feature Scaling*:
  - Performance depends on the proper scaling of features, as different scales can distort distance calculations.

K-Nearest Neighbors (KNN)

- **Historical Context**: Introduced in the 1950s by Fix and Hodges, formalized in the 1960s by Cover and Hart, KNN was initially popular for pattern recognition and statistical estimation due to its simplicity and intuitive approach.

K-Nearest Neighbors(KNN)

- K-Nearest Neighbors (KNN) is a simple, non-parametric, and lazy learning algorithm used for classification and regression tasks. KNN is an instance-based learning algorithm that classifies a data point based on how its neighbors are classified.



- Algorithm Steps:
  - **Select K**: Choose the number of neighbors (K).
  - **Calculate Distance**:Compute the distance between the new data point and all the training points (commonly used distance metrics are Euclidean Manhattan).
  - **Find Nearest Neighbors**: Identify the K nearest neighbors to the new data point.
  - **Vote (for classification)**:Each neighbor votes for their class, and the class with the most votes is assigned to the data point.
  - **Average (for regression)**: The average value of the K nearest neighbors is taken as the prediction.

Euclidean Distance: $\sqrt{\sum (x_i - y_i)^2}$

Manhattan Distance: $\sum |x_i - y_i|$

- To classify the new data point (170, 57) using the K-Nearest Neighbors (KNN) algorithm and Euclidean distance, follow these steps

Calculate the Euclidean Distance:

- The Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Compute Distances for All Points:

- Calculate the distance between the new point (170, 57) and each point in the dataset.

Select the Nearest Neighbors:

- Choose the $K = 3$ nearest neighbors.

Height(cm)	Weight(kg)	Classification
168	50	Underweight
181	63	Normal
175	68	Normal
172	63	Normal
173	64	Normal
173	55	Underweight
170	59	Normal
174	56	Normal
170	57	?

Distance to (168, 50):

$$d = \sqrt{(170-168)^2 + (57-50)^2} = \sqrt{2^2 + 7^2} = \sqrt{4+49} = \sqrt{53} \approx 7.28$$

Distance to (181, 63):

$$d = \sqrt{(170-181)^2 + (57-63)^2} = \sqrt{(-11)^2 + (-6)^2} = \sqrt{121+36} = \sqrt{157} \approx 12.53$$

Distance to (175, 68):

$$d = \sqrt{(170-175)^2 + (57-68)^2} = \sqrt{(-5)^2 + (-11)^2} = \sqrt{25+121} = \sqrt{146} \approx 12.08$$

Distance to (172, 63):

$$d = \sqrt{(170-172)^2 + (57-63)^2} = \sqrt{(-2)^2 + (-6)^2} = \sqrt{4+36} = \sqrt{40} \approx 6.32$$

Distance to (173, 64):

$$d = \sqrt{(170-173)^2 + (57-64)^2} = \sqrt{(-3)^2 + (-7)^2} = \sqrt{9+49} = \sqrt{58} \approx 7.62$$

Distance to (173, 55):

$$d = \sqrt{(170-173)^2 + (57-55)^2} = \sqrt{(-3)^2 + 2^2} = \sqrt{9+4} = \sqrt{13} \approx 3.61$$

Distance to (170, 59):

$$d = \sqrt{(170-170)^2 + (57-59)^2} = \sqrt{0^2 + (-2)^2} = \sqrt{4} = 2$$

Distance to (174, 56):

$$d = \sqrt{(170-174)^2 + (57-56)^2} = \sqrt{(-4)^2 + 1^2} = \sqrt{16+1} = \sqrt{17} \approx 4.12$$

Height(cm)	Weight(kg)	Classification
168	50	Underweight
181	63	Normal
175	68	Normal
172	63	Normal
173	64	Normal
173	55	Underweight
170	59	Normal
174	56	Normal
170	57	?

Nearest Neighbors:

- The distances are:
  - (170, 59): 2 (Normal)
  - (173, 55): 3.61(Underweight)
  - (174,56): 4.12 (Normal)

- Classification:
  - The majority class among the nearest neighbors is"Normal".
  - Thus, the new data point (170, 57) is classified as "Normal".

- *Advantages:*
  - Simple and easy to implement.
  - No training phase (lazy learning), which makes it fast for small data sets.

- *Disadvantages:*
  - Computationally expensive for large data sets.
  - Sensitive to the choice of K and the distance metric.
  - Poor performance with imbalanced data.

- *Applications:*
  - Pattern recognition
  - Data mining
  - Image classification
  - Recommendation systems

- **Modern-Day Use**: KNN is currently applied in image recognition, recommendation systems, anomaly detection, and genomics. It remains favoured for its ease of implementation and versatility, despite computational challenges, which are mitigated by techniques like KD-Trees, PCA, and distributed computing.

Current Research and Trends: Ongoing research focuses on optimizing KNN for specific applications and enhancing efficiency through approximate nearest neighbour searches. Trends include integrating KNN with deep learning models, using it in edge computing and IoT, and improving performance with parallel processing and GPU acceleration.